1. Why Do We Need Volumes?

In real Data Engineering projects, data does not always arrive as a table.

Source systems may send:

- CSV files
- JSON files
- Parquet files
- Images
- Text files
- Other file formats

Example:

orders.csv

Before converting this data into a Delta table, we need a governed place to store and access the file.

That is where **Unity Catalog Volumes** are useful.

---

2. What Is a Unity Catalog Volume?

A Unity Catalog Volume is a governed storage object used to manage and access files through Unity Catalog.

Simple definition:

> A Volume provides governed file-level access inside Databricks.

A Volume can contain files such as:

CSV
JSON
Parquet
Images
Text files

Volumes are useful when we want to work directly with files rather than registered tables.

---

3. Tables vs Volumes

This distinction is very important.

Unity Catalog Table

Used for structured records.

Example:

datax_catalog.bronze.orders

We can query it using:

```
SELECT *
FROM datax_catalog.bronze.orders;
```

Tables are useful for:

- Structured data
- SQL queries
- PySpark DataFrames
- Delta operations such as MERGE and UPDATE

Unity Catalog Volume

Used to access and manage files.

Example path:

/Volumes/datax_catalog/bronze/orders_files/

Useful for:

- CSV
- JSON
- Parquet files
- Images
- Other files

We do **not** use:

```
SELECT *
FROM volume_name;
```

Instead, we read files using their path.

Easy Way to Remember

TABLE
→ Structured Records

VOLUME
→ Files

---

4. How Tables and Volumes Work Together

A common ingestion pattern can be:

Source Application
↓
orders.csv
↓
Unity Catalog Volume
↓
Spark DataFrame
↓
Delta Table

The Volume provides governed access to the original file.

The table provides structured, queryable data.

---

5. Managed vs External Volumes

Unity Catalog supports two types:

Managed Volume
External Volume

The biggest difference is:

> Who manages the underlying storage location and lifecycle?

---

6. Managed Volume

A managed volume:

- Uses Unity Catalog-managed storage
- Uses configured managed storage
- Is suitable for files managed inside the Databricks platform
- Has its data lifecycle tied to the managed volume

Conceptually:

Unity Catalog
↓
Managed Volume
↓
Managed Storage

Dropping the volume can also affect the underlying managed data according to the managed-data lifecycle.

---

7. External Volume

An external volume:

- Points to an existing cloud storage location
- Can point to ADLS Gen2 in Azure
- Requires an external location
- Requires appropriate permissions
- Keeps the underlying external files even if the UC volume object is dropped

Conceptually:

Unity Catalog
↓
External Volume

↓
Existing Cloud Storage

Example use case:

> Your company already has an ADLS Gen2 folder where incoming order files are stored.

An external volume can provide governed access to those files.

---

8. Managed vs External Volume — Quick Comparison

Managed Volume	External Volume
UC-managed storage	Existing external storage
Storage lifecycle managed with volume	Storage lifecycle independent
Simpler for Databricks-managed files	Useful for existing cloud locations
Underlying data can be affected on drop	External files remain

---

9. Where Do Volumes Exist?

Volumes exist inside:

Catalog
↓
Schema
↓
Volume

Example:

datax_catalog
↓
bronze

↓
orders_files

So a volume belongs to both:

- Catalog
- Schema

---

10. Create a Managed Volume

Create:

```
CREATE VOLUME IF NOT EXISTS
datax_catalog.bronze.orders_files;
```

Because no external location is specified, this creates a **managed volume**.

Validate:

```
SHOW VOLUMES IN datax_catalog.bronze;
```

Expected:

orders_files

Creating a volume requires appropriate permissions and usable managed storage.

---

11. Unity Catalog Volume Path

Volumes use a standard Databricks path structure:

/Volumes/catalog/schema/volume/

Example:

/Volumes/datax_catalog/bronze/orders_files/

Breakdown:

/Volumes/

→ Volume path prefix

datax_catalog
→ Catalog

bronze
→ Schema

orders_files
→ Volume

---

12. Important: `/Volumes/` Is Not the Physical ADLS Path

Do not assume:

/Volumes/datax_catalog/bronze/orders_files/

is the actual cloud-storage folder structure.

It is a governed Databricks path.

Unity Catalog maps that path to the underlying configured storage.

---

13. Access a Volume Using PySpark

Define:

volume_path = "/Volumes/datax_catalog/bronze/orders_files/"

List files:

```
display(
    dbutils.fs.ls(volume_path)
)
```

This allows us to work with volume files using supported Databricks APIs.

---

14. Upload a CSV File

Upload:

orders.csv

into:

datax_catalog
→ bronze
→ orders_files

After upload:

```
display(
    dbutils.fs.ls(volume_path)
)
```

You should see:

orders.csv

At this stage:

We have stored a file
≠
We have created a Delta table

This is just the source file.

---

15. Read CSV from the Volume

File path:

```
orders_path = (
    "/Volumes/datax_catalog/bronze/"
    "orders_files/orders.csv"
)
```

Read:

```
orders_df = (
    spark.read
    .option("header", "true")
    .csv(orders_path)
)

display(orders_df)
```

We can read the file directly without creating a table first.

---

16. Why All Columns May Appear as Strings

Run:

```
orders_df.printSchema()
```

When reading CSV without an explicit schema, columns may appear as strings.

This is because CSV does not provide the same strong typing as a structured table.

For production pipelines, relying entirely on automatic inference can create problems.

---

17. Define an Explicit Schema

A more reliable approach is to define the expected schema.

Example:

```
from pyspark.sql.types import (
    StructType,
    StructField,
    IntegerType,
    StringType,
    DateType,
    DecimalType
```

)

Schema:

```
orders_schema = StructType([
    StructField("order_id", IntegerType(), True),
    StructField("customer_id", StringType(), True),
    StructField("product_id", StringType(), True),
    StructField("order_date", DateType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("unit_price", DecimalType(10, 2), True),
    StructField("order_status", StringType(), True)
])
```

---

18. Read Using Explicit Schema

```
orders_df = (
    spark.read
    .option("header", "true")
    .schema(orders_schema)
    .csv(orders_path)
)

display(orders_df)
```

Then:

```
orders_df.printSchema()
```

Now the expected data types are applied.

Why This Is Better

Explicit schemas provide:

- More predictable pipelines
- Consistent data types
- Less risk of incorrect inference

But explicit schema alone does not guarantee that the records are valid.

---

19. Basic Data Validation

After ingestion, perform basic checks.

Record Count

```
print(
    "Total Orders:",
    orders_df.count()
)
```

Expected:

10

Null Checks

Example:

```
from pyspark.sql.functions import col

display(
    orders_df.filter(
        col("order_id").isNull()
        | col("customer_id").isNull()
        | col("quantity").isNull()
    )
)
```

For the sample dataset, no rows are expected.

---

20. Status Validation

Check different order statuses:

```
display(
    orders_df
    .groupBy("order_status")
    .count()
```

)

This helps us understand how many records are:

COMPLETED
PENDING
CANCELLED

These are basic validation checks.

---

21. Invalid Records in Production

If invalid records appear, possible actions include:

Reject
Quarantine
Correct
Send for Review

The correct action depends on business requirements.

---

22. File Metadata for Traceability

Suppose you ingest:

100 CSV files every day

Later, a business user reports a bad record.

A useful question is:

> Which source file did this record come from?

This is where file metadata becomes useful.

---

23. Capture Source File

During ingestion, capture:

source_file

using:

_metadata.file_path

Also capture:

ingestion_timestamp

Example:

```
from pyspark.sql.functions import (
    col,
    current_timestamp
)
```

Then:

```
orders_with_metadata_df = (
    spark.read
    .option("header", "true")
    .schema(orders_schema)
    .csv(orders_path)
    .select(
        "*",
        col("_metadata.file_path").alias("source_file")
    )
    .withColumn(
        "ingestion_timestamp",
        current_timestamp()
    )
)
```

---

24. Why File Metadata Matters

Now each record can contain:

Business Columns
+
source_file

+
ingestion_timestamp

This helps with:

- Debugging
- Auditing
- Traceability
- Identifying problematic source files
- Production troubleshooting

---

25. How Volumes Fit Into a Real Pipeline

A possible production architecture:

E-commerce Application
↓
orders.csv
↓
Cloud Storage / Volume
↓
Bronze
↓
Silver
↓
Gold

---

26. Bronze Layer

Bronze can:

- Read incoming files
- Preserve source metadata
- Store ingested records
- Keep data close to its original form

Example:

Source CSV

↓
Volume
↓
Bronze Delta Table

---

27. Silver Layer

Silver can:

- Validate required columns
- Handle invalid data
- Deduplicate records
- Apply transformations
- Clean the data

Example:

Bronze
↓
Validation
↓
Deduplication
↓
Clean Silver Data

---

28. Gold Layer

Gold contains business-ready datasets.

Examples:

Daily Sales
Revenue by Product
Reporting Datasets
Analytics Tables

---

29. Do All Pipelines Need Volumes?

No.

Volumes are useful for governed file access, but some pipelines may:

- Read directly from supported cloud storage
- Use external locations
- Use other managed ingestion features

The architecture depends on:

- How files arrive
- Storage requirements
- Governance requirements
- Ingestion design

---

30. Common Mistakes

Mistake 1 — Confusing Volume With Table

Volume
→ Files

Table
→ Structured records

Mistake 2 — Assuming `/Volumes/` Is the Physical ADLS Path

It is a governed Databricks path.

Mistake 3 — Using Schema Inference Everywhere

Production pipelines should usually define and validate expected types.

Mistake 4 — Giving Everyone Write Permission

Follow least privilege.

Users who only need to read files should not automatically receive write access.

Mistake 5 — Dropping Managed Volumes Without Understanding Lifecycle

Managed and external volumes behave differently when dropped.

Always understand what will happen to the underlying data.

---

31. Quick Revision

Unity Catalog
↓
Catalog
↓
Schema
↓
Volume
↓
Files

Example:

/Volumes/datax_catalog/bronze/orders_files/

---

32. Table vs Volume — Quick Revision

Table	Volume
Structured records	Files
SQL/DataFrame access	File-path access

Can support Delta operations	Used for file management
Example: `bronze.orders`	Example: `/Volumes/.../orders_files/`

---

33. Managed vs External Volume — Quick Revision

MANAGED VOLUME
→ Managed storage
→ Managed lifecycle

EXTERNAL VOLUME
→ Existing cloud storage
→ Explicit external location
→ External files remain independently managed

---

34. Student Practice Assignment

Task 1 — Create a Managed Volume

Create:

datax_catalog.bronze.practice_files

using:

CREATE VOLUME IF NOT EXISTS
datax_catalog.bronze.practice_files;

Task 2 — Upload `orders.csv`

Upload the existing:

orders.csv

into the volume.

Verify using Catalog Explorer.

Task 3 — Read Using PySpark

Read from:

/Volumes/

Then:

- Display records
- Print schema

Task 4 — Define Explicit Schema

Use:

StructType
StructField

Ensure:

order_date → DateType
unit_price → DecimalType

Task 5 — Data Validation

Perform:

- Total record count
- Null check for `order_id`
- Group by `order_status`

Task 6 — File Metadata

Add:

source_file
ingestion_timestamp

Then explain how these columns improve troubleshooting.

---

35. Interview-Relevant Questions

1. What is a Unity Catalog Volume?

A Unity Catalog Volume is a governed storage object used to access and manage files through Unity Catalog.

---

2. What is the difference between a table and a volume?

A table contains structured records that are queried using SQL or DataFrames.

A volume is used for direct governed file access.

---

3. What is the difference between managed and external volumes?

Managed volumes use Unity Catalog-managed storage and lifecycle.

External volumes reference existing cloud storage and do not delete underlying external files when the volume object is dropped.

---

4. What is the standard Unity Catalog Volume path?

/Volumes/catalog/schema/volume/

---

5. Can we query a Volume using `SELECT * FROM volume_name`?

No.

A volume is not a table.

Files inside it are accessed using paths and file APIs.

---

6. Why use an explicit schema when reading CSV?

It makes expected data types predictable and reduces incorrect automatic inference.

Additional data validation is still necessary.

---

7. How can you find which source file produced a record?

Capture:

_metadata.file_path

during file ingestion.

---

8. Why capture ingestion timestamps?

They help identify when a record entered or was processed by the pipeline.

---

9. When would you use an external volume?

When files already exist in an external cloud-storage location such as ADLS Gen2 and we want governed access through Unity Catalog.

---

10. Do all ingestion pipelines require a Volume?

No.

Volumes are one approach for governed file access, but other ingestion architectures are also possible.

Key Takeaway

Source System
↓
Raw CSV / JSON / Files
↓
Unity Catalog Volume
↓
PySpark Read
↓
Explicit Schema
↓
Validation
↓
Source Metadata
↓
Bronze Delta Table
↓
Silver
↓
Gold

The most important idea from this class is:

Unity Catalog Tables govern structured datasets, while Unity Catalog Volumes provide governed access to files used in Data Engineering pipelines.